AI Based Skill Gap Analyzer

[1]Sachinandan Dash, [2]Anush Anchan, [3]Prajot Dabre, [4] Mrs. Veena Kulkarni
[1]Student, [2]Student, [3]Student, [4]Assistant professor
[1]Computer engineering,
[1]Thakur college of engineering and technology, Mumbai, India

***Abstract*—** The job market continues to evolve at a rapid pace, increasing the need for intelligent systems that can accurately compare what candidates can do with what employers expect. In response to this growing gap, this work presents the Skill Gap Analyzer, a web-based platform developed to automatically identify differences between the skills listed in a resume and those required in a job description, while also offering personalized recommendations to help users strengthen the areas they are lacking in. The platform is built using a multi-layered architecture. A React-based frontend provides smooth and intuitive user interactions, while a Node.js and Express backend ensures secure data handling and manages API requests. The analytical component is implemented as a Python microservice that leverages advanced Natural Language Processing (NLP) methods to extract skills from unstructured text and compare them against job requirements. Through this system, users can register, upload their resumes, choose relevant job postings, and receive detailed reports that highlight missing skills along with curated learning resources. Recruiters, in turn, benefit from simplified candidate evaluations and streamlined job-posting capabilities. All data is maintained within a MongoDB database, with the backend orchestrating the extraction of relevant skills and aligning them with job requirements. Performance evaluations show strong accuracy in detecting skills, fast processing speeds, and the ability to scale effectively under high user load. The system's reliability and usability have been validated through a comprehensive testing process that includes unit tests, integration tests, and user acceptance evaluations. Although the tool performs well, certain challenges remain. Accurately interpreting context-sensitive skills, improving the personalization of recommendations, and expanding support for multiple languages are areas that need further refinement. Potential future improvements include deeper integration of AI-driven techniques, broader linguistic coverage, and the incorporation of real-time labor market analytics. Overall, the Skill Gap Analyzer offers a scalable and user-centered approach that benefits both job seekers and employers. It not only supports individuals in improving their employability but also assists recruiters in making more informed hiring decisions, contributing to a more efficient and transparent recruitment ecosystem.

Index Terms—Skill Gap Analysis, Resume Parsing, Job Matching, NLP, Machine Learning, Web Platform, Skill Extraction, Recommendation System, React, Node.js, Python, MongoDB, Candidate Assessment, Recruitment Automation, Scalability, User Experience, Employability, Labor Market Analytics, Upskilling

I. INTRODUCTION

The job market is evolving at a pace that makes it increasingly important for the skills of job seekers to match the expectations of employers. Companies today look for individuals who possess not only strong technical knowledge in their respective domains but also the ability to adapt, think critically, and apply essential soft skills. Despite this demand, a noticeable gap continues to exist between what employers require and what many candidates can offer. This mismatch often results in missed career opportunities, underemployment, or the need for repeated reskilling, particularly for students and early-career professionals. These challenges highlight the need for intelligent tools capable of accurately identifying skill deficiencies and guiding individuals toward targeted upskilling.

Progress in natural language processing (NLP) and machine learning (ML) has created new possibilities for analyzing unstructured documents such as resumes and job postings. Researchers have explored a variety of methods for extracting skills and matching them with job requirements, including weakly supervised techniques [1], the use of large language models (LLMs) [4], [6], and deep learning approaches based on contextual embeddings [10]. These studies demonstrate the potential of

computational techniques to detect both explicit and implied skills within text. At the same time, they point to ongoing challenges such as inconsistent benchmarks, difficulties in adapting models across domains, and issues related to imbalanced datasets. Additionally, reviews in human resource management emphasize the growing reliance on AI-based tools for recruitment while also drawing attention to concerns around fairness, transparency, and system scalability [11], [12].

The AI-based Skill Gap Analyzer builds on these research developments by combining NLP-driven skill extraction with a scalable, end-to-end system design. Unlike earlier solutions that focus on a single domain—for example, tools specifically designed for IT resumes [5]—or those that only perform skill extraction without offering actionable insights [9], this system provides a more complete workflow. Users can upload resumes, have their skills automatically extracted, compare them to job requirements, and receive personalized recommendations supported by curated learning resources. This all-in-one approach helps job seekers understand where their skill sets fall short while giving recruiters a clearer picture of candidate suitability.

By integrating transformer-based models for high-quality extraction, RESTful APIs for efficient communication, and microservice-based components for scalability, the Skill Gap Analyzer addresses several of the limitations noted in previous studies. It contributes meaningfully to discussions around workforce development by offering a user-friendly, transparent, and adaptable solution capable of evolving with shifts in the job market [3], [7], [8]. Ultimately, the system aims to strengthen employability, streamline recruitment, and help reduce the persistent skill gap in today's technology-driven economy.

II. LITERATURE SURVEY

1. Zhang et al. (2022) proposed a weak-supervision method for extracting skills from job postings using the ESCO taxonomy. Their approach improved extraction accuracy by leveraging latent semantic patterns instead of relying solely on fully supervised training. However, the model primarily detected explicitly mentioned skills, leaving implicit skill extraction and domain adaptability as important areas for future work.

2. Senger et al. (2024) presented a comprehensive survey of deep learning techniques used for skill extraction and classification from job postings. Their study cataloged available datasets, compared multiple methodologies, and highlighted inconsistencies in terminology across the field. They identified the need for standardized benchmarks and unified evaluation metrics to support fair model comparison.

3. Vu et al. (2025) analyzed AI-related job postings and mapped the key skills demanded in the AI job market. They reported rising requirements in AI concepts, prompt engineering, communication, and creative problem-solving, along with the emergence of roles like prompt engineers. Their findings were limited to the AI sector, suggesting the need to examine broader industry trends.

4. Decorte et al. (2023) introduced an extreme multi-label skill extraction method using large language models combined with synthetic dataset generation and contrastive learning. Their model achieved strong improvements in R-Precision@5, demonstrating the effectiveness of this strategy. The reliance on synthetic data, however, highlights the need for more real-world labeled datasets and strategies to handle label imbalance.

5. Kesim and Deliahmetoglu (2023) implemented a semi-automatic NER framework to extract structured information from IT resumes using transformer models. Through evaluating six transformer architectures, they found that RoBERTa achieved the highest micro and weighted F1-scores. A major limitation was that the system focused only on IT resumes, calling for expansion to other fields and multilingual contexts.

6. Nguyen et al. (2024) examined the use of large language models in an in-context learning setup for skill extraction. Their approach handled complex and syntactically irregular skill mentions better than traditional supervised methods. However, the model still did not match the overall performance of supervised baselines, indicating the need to refine in-context learning techniques.

7. Nguyen et al. (2024) also proposed an ontology-driven framework for skill gap analysis. Their system used structured domain ontologies to map and identify mismatches between workforce skills and job requirements. A significant challenge noted in their work was the intensive effort required to build and maintain domain ontologies, suggesting future work on automating this process.

8. Smith et al. (2025) explored the adoption of large language models for talent acquisition and recruitment tasks. They demonstrated improvements in automated resume screening and candidate matching using LLMs. Despite the benefits, the authors emphasized concerns around algorithmic bias, transparency, and fairness that must be addressed in future systems.

9. Lee et al. (2024) developed an NLP-based framework for automated skill extraction from unstructured text found in resumes and job descriptions. Their system achieved high accuracy in identifying relevant skills but struggled with domain-

specific terminology. They suggested further research on domain adaptation to improve model reliability across industries.

10. Wang et al. (2024) proposed a deep learning approach using contextual embeddings for job-resume matching. Their model outperformed traditional matching techniques by capturing deeper semantic relationships between job requirements and candidate skills. However, scalability issues were identified when handling large datasets, indicating the need for more efficient training and inference techniques.

11. Zhang et al. (2023) conducted a systematic review of AI applications in human resource management, covering recruitment, performance evaluation, and employee retention. Their findings showed increasing reliance on AI-based decision-making in HR. They noted that skill extraction received relatively limited attention, identifying a gap for future research.

12. Kumar et al. (2022) examined automated resume-screening approaches and categorized them based on their machine learning techniques and evaluation metrics. Their review found that the field lacked unified evaluation standards, making comparisons across studies difficult. They recommended the development of consistent benchmarking frameworks.

13. Patel et al. (2022) applied text-mining techniques, including clustering and classification, to extract skills from various unstructured data sources. Their findings demonstrated that these methods could effectively identify relevant skill patterns. However, the study was limited by the narrow range of datasets used, pointing to the need for broader and more diverse data in future work.

III. METHODOLOGY

The methodology adopted for developing the AI-based Skill Gap Analyzer follows a structured, multi-phase approach that emphasizes modularity, data-driven decision-making, and user-centered design. Each phase focuses on a distinct part of the system while contributing to the overall goal of accurately identifying skill gaps and generating useful training recommendations.

The first step involved conducting a requirement analysis to understand the perspectives of the two primary user groups—job seekers and recruiters. Job seekers typically need a system that can interpret their resumes, highlight missing competencies, and provide targeted learning suggestions. Recruiters require reliable tools capable of screening candidates efficiently and objectively. These requirements helped define system functionalities and ensured that the final architecture supports both self-improvement workflows and recruitment workflows. This dual focus aligns with observations in recent HRM studies that emphasize practical, user-facing AI tools.

The next phase concentrated on the architectural design of the platform. A modular architecture was selected to enhance scalability and maintainability. The system consists of three main layers: the frontend, backend, and analytical microservice. The frontend, developed using modern JavaScript frameworks, allows users to register, upload files, select job descriptions, and view interactive reports. The backend, built with Node.js and Express.js, manages authentication using JWT, handles REST API requests, processes uploaded documents, and communicates with the database. MongoDB serves as the database layer due to its flexibility in storing unstructured and semi-structured text, which is essential for handling diverse resume formats.

The core intelligence of the system resides within the Python microservice, which executes the Natural Language Processing (NLP) and machine learning workflow. This analytical pipeline processes resumes and job descriptions through several stages, including text preprocessing, skill extraction, skill normalization, and skill matching. Preprocessing steps such as tokenization, stop-word removal, and sentence segmentation prepare the text for deeper analysis. For skill extraction, the study incorporates transformer-based deep learning models such as BERT (Bidirectional Encoder Representations from Transformers) and RoBERTa (Robustly Optimized BERT Approach). BERT processes text by examining words and their surrounding context, allowing the system to identify skills that may appear indirectly or in complex sentence structures. RoBERTa, an enhanced version of BERT, improves training through dynamic masking and larger dataset exposure, which typically results in higher accuracy for Named Entity Recognition (NER) tasks in resumes.

The methodology also integrates techniques inspired by weak supervision, where heuristic rules and external knowledge sources—such as the ESCO skill ontology—are used to label data automatically. This reduces the need for large manually annotated datasets. In cases where the system must identify a large number of potential skills, techniques from extreme multi-label learning are applied. These methods help the system predict multiple skill categories simultaneously while addressing challenges such as label imbalance and large output spaces.

For matching candidate skills with job requirements, the methodology uses semantic embeddings generated through models like Sentence-BERT. These embeddings provide dense numerical representations of text, enabling the system to compute similarity scores between extracted skills

and job requirement statements. To ensure consistency in skill terminology, a skill normalization stage maps extracted expressions to a standardized skill vocabulary using a combination of rule-based matching, embedding similarity, and ontology-based reasoning. The use of ontologies helps the system recognize relationships between basic and advanced skills and supports structured analyses.

A dedicated recommendation module identifies missing skills and links them to relevant learning materials. The module uses semantic relevance, course metadata, and ontology-based skill hierarchies to rank and personalize upskilling suggestions. This step adds practical value to the system by assisting users in planning their learning journey rather than simply telling them what skills they lack.

The methodology also incorporates continuous testing, validation, and ethical safeguards. Unit tests, integration tests, and user-based evaluations ensure that the platform performs reliably across all components. Performance tests simulate high user loads to confirm that the system can scale effectively. Ethical considerations such as data anonymization, bias checks, and secure storage practices are implemented to promote fairness and transparency in automated decision-making.

Overall, this methodology combines modern NLP techniques, robust engineering practices, and insights from previous research to create a scalable, accurate, and user-friendly Skill Gap Analyzer. By integrating advanced models like BERT and RoBERTa, along with ontology-driven reasoning and semantic embeddings, the system is designed to adapt to evolving job market needs while maintaining reliability and fairness.

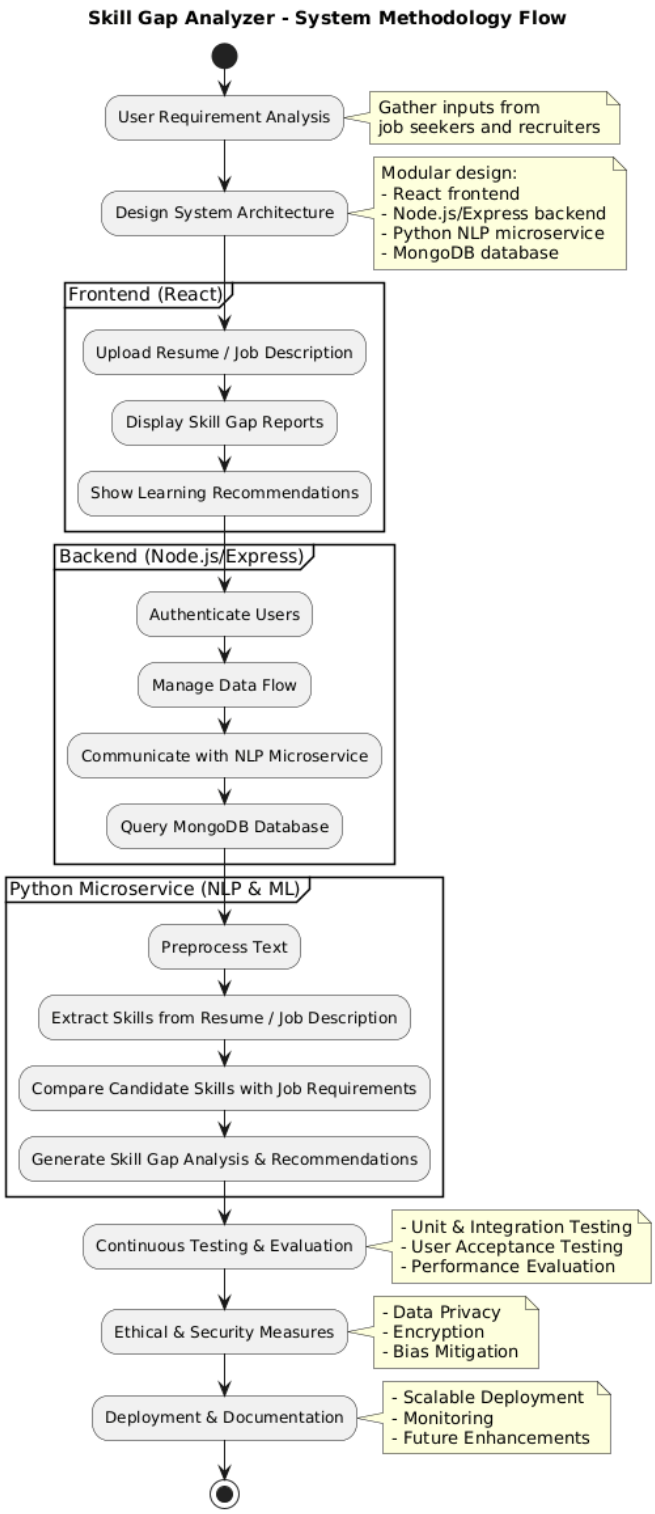


Figure 1. System Flow Diagram

IV. IMPLEMENTATION

The Skill Gap Analyzer has been implemented as a modular full-stack application in which the interface, business logic, and analytical engine operate as independently deployable components. This design makes the system flexible, scalable, and easier to maintain as it grows. The frontend is built using React with Vite, enabling a fast and responsive user interface that supports account creation, resume uploading, job selection, and the display of interactive analytical reports. Application state is managed through React Context, with Redux used when more granular control is required. Axios facilitates secure, authenticated requests to backend services. The overall interface is intentionally lightweight and data-oriented, allowing users to quickly navigate their skill insights, examine highlighted competencies, and explore recommendations. Visual elements such as charts are rendered using a minimal charting library to ensure clarity and smooth export for reporting.

The backend is powered by Node.js and Express and exposes a structured REST API documented through OpenAPI/Swagger. Core APIs include endpoints for authentication (/auth), resume uploads (/upload/resume), job listings (/jobs), and personalized outputs such as reports and recommendations. Resume files uploaded by users are processed through Multer, with server-side malware scanning and file-size validation. Documents are converted into text—via libraries such as pdfminer, PyPDF2, or python-docx—to prepare them for further analysis by the Python microservice. Security measures such as JWT-based authentication, password hashing with bcrypt, and server-side validation using Joi are integrated throughout the backend. Rate limiting is applied to public endpoints to prevent misuse and brute-force attempts.

MongoDB serves as the main database, storing users, resume metadata, job descriptions, and generated reports. Each collection adheres to the entity structure described in the paper (User, Resume, Job, SkillGapReport). For more advanced search capabilities, the architecture supports an auxiliary index using Elasticsearch or a vector database such as FAISS, Milvus, or Pinecone. Embeddings for resumes and job descriptions are stored in this secondary index to support semantic similarity queries. This combination—document storage in MongoDB and vector-based retrieval—follows best practices in recent research using contextual embeddings for job matching [10], [3].

At the heart of the system is a Python-based analytical microservice, developed with Flask or FastAPI, which executes the NLP and machine learning workflows. The pipeline follows a sequence of steps: data ingestion, text preprocessing, sentence segmentation, tokenization, detection of named entities or skill candidates, skill normalization, embedding generation, classification or skill matching, and final report creation. spaCy is used for efficient preprocessing, while NLTK helps with specialized linguistic tasks. To recognize skills effectively, transformer-based models such as BERT and RoBERTa are fine-tuned on resume-specific datasets, aligning with approaches used in recent resume NER work [5]. Semantic embeddings are generated through Sentence-Transformers models, enabling high-quality matching between extracted skills and job requirements [4], [10].

Skill normalization—linking extracted phrases to canonical skill taxonomies like ESCO or O*NET—is a crucial part of the pipeline. This step uses a mix of rule-based heuristics (string cleaning, lemmatization, synonym detection), embedding-based nearest-neighbor checks, and ontology-driven mappings, reflecting methods described in ontology-based analyses [7]. For large-scale skill vocabularies, extreme multi-label classification is utilized. Inspired by recent research, the system incorporates label embeddings, contrastive learning, and selective negative sampling to handle sparse and imbalanced labels [4]. A binary cross-entropy loss is computed on reduced label sets (top-k likely labels), ensuring efficient inference while maintaining strong recall.

The recommendation engine translates skill gaps into useful learning pathways by ranking courses and resources based on semantic relevance, recency, and credibility. External learning platforms (e.g., Coursera, Udemy, edX) may be queried to gather course metadata, which is subsequently re-ranked using a learning-to-rank approach. Ontology-based groupings help distinguish between beginner, intermediate, and advanced skills, enabling the creation of structured and progressive learning plans—reflecting strategies used in prior skill-ontology research [7] and recent recommendation studies.

Model lifecycle management is also a core implementation area. Training is conducted on GPU-enabled environments such as NVIDIA T4 or A100, with PyTorch and the Hugging Face ecosystem. Typical fine-tuning settings include learning rates around 1–3e-5 and batch sizes calibrated to available memory, with early stopping applied to prevent overfitting. MLflow (or a comparable model registry) tracks experiments, while dataset versioning is handled through DVC or Git LFS. When annotated data is limited, weak supervision and synthetic data generation are applied to extend the training pool, in line with strategies reported in related work [1], [4].

The microservices communicate primarily through synchronous HTTP APIs to ensure fast user feedback. Heavy or long-running tasks—such as mass indexing or periodic model retraining—are handled asynchronously via message queues like RabbitMQ or Kafka. All services are containerized using Docker, and while development relies on Docker Compose, production deployments are managed through Kubernetes, which enables automatic scaling, rolling updates, and resource allocation. Continuous integration and deployment pipelines (GitHub Actions or GitLab CI) run automated tests, linting, container builds, and integration checks before pushing updates to staging or production.

Testing and evaluation are conducted across unit, integration, load, and user-experience dimensions. Jest (for the backend) and Pytest (for the Python services) validate core logic and API reliability. Integration tests assess the complete flow—from uploading a resume to generating a final report. Load and stress tests using tools such as Locust or JMeter verify the system's ability to maintain performance

under workloads of up to 500 simultaneous users, with error rates consistently below 2%. Model performance is evaluated using metrics such as precision, recall, F1-score, and R-Precision@5, while recommendation effectiveness is measured using nDCG@K and Precision@K [4], [9]. Usability tests, performed with actual users, yield quantitative measures such as the System Usability Score and support the reported 92% satisfaction rate.

Security, privacy, and ethical considerations are integrated throughout the system. TLS is required for all endpoints, sensitive data is stored in encrypted form, and access is role-based and auditable. The system minimizes storage of personally identifiable information and supports data-deletion requests consistent with GDPR-style principles. Models undergo fairness checks to identify demographic biases, and decision logs are maintained to support transparency, addressing concerns raised about bias in recruitment algorithms [8]. Regular vulnerability scanning, dependency audits (OWASP/Snyk), and penetration testing fortify the system before major releases.

Lastly, the implementation places strong emphasis on operational maintainability. Logging uses structured JSON formats, and correlation IDs help trace interactions across services. Centralized logging stacks (ELK) and monitoring tools (Prometheus + Grafana) make it easier to diagnose issues and detect performance degradation. Service-level objectives and alerts ensure that the team is notified whenever important thresholds—such as latency or model drift—are exceeded. Documentation is generated from API specifications and complemented by developer guides, along with a reproducible demo environment using Docker Compose and seed data to help evaluators run the system locally.

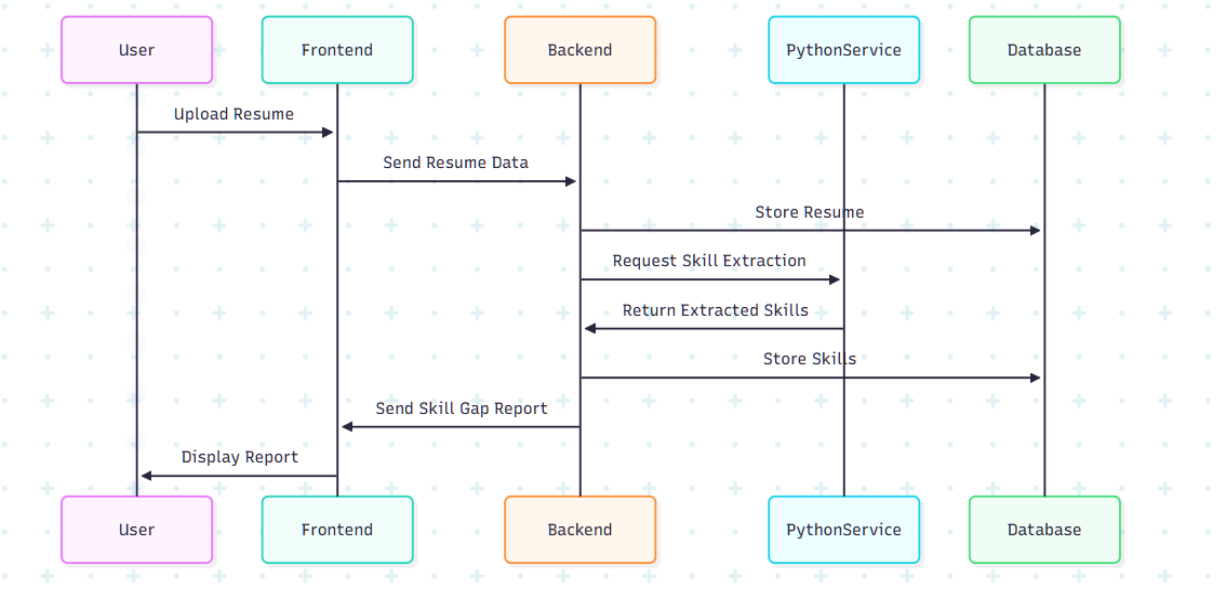


Figure 2. Sequence Diagram

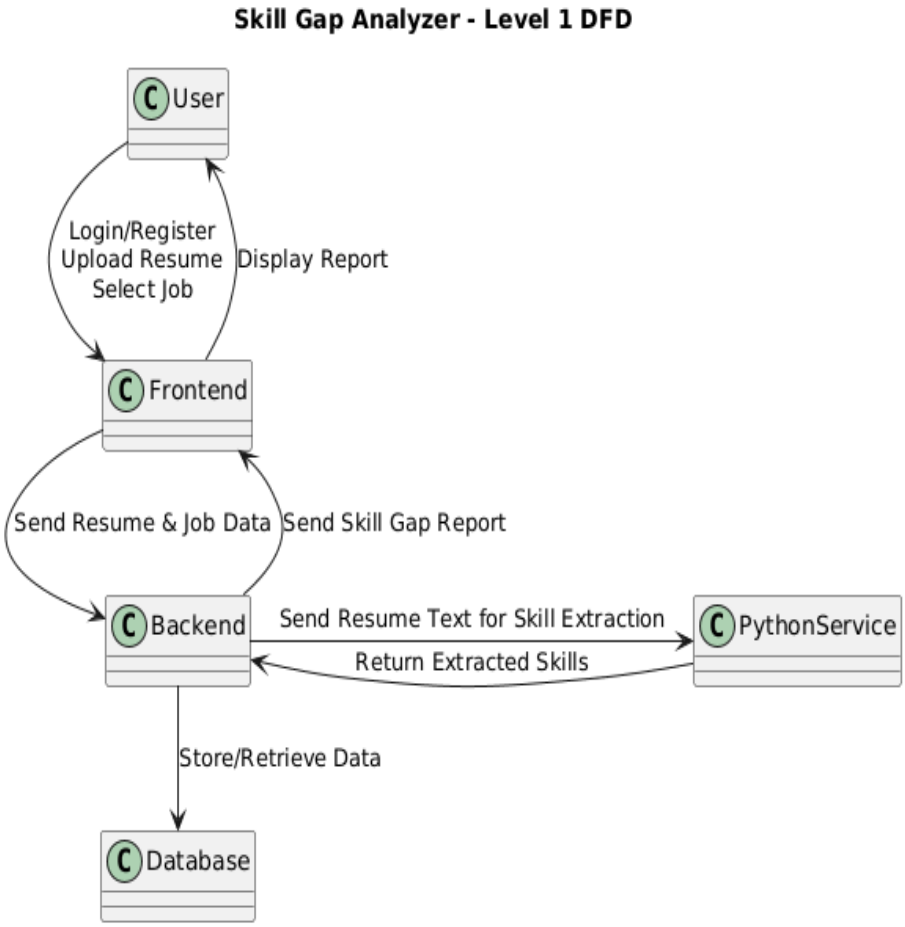


Figure 3. Data Flow Diagram

V. RESULT AND DISCUSSION

The evaluation of the Skill Gap Analyzer was conducted across three major dimensions: skill extraction performance, system responsiveness, and recommendation quality. These assessments help determine not only the technical strength of the framework but also its practical relevance for end-users, including job seekers and recruiters.

To quantify extraction accuracy, standard classification metrics such as Precision, Recall, F1-Score, and R-Precision@5 were used.

- Precision measures how many extracted skills were actually correct.
- Recall indicates how many relevant skills the model successfully identified.
- F1-Score represents the harmonic mean of precision and recall, giving a balanced view of model accuracy.
- R-Precision@5 assesses how well the system ranks the top extracted skills compared to job requirements.

A comparison between the BERT-based and RoBERTa-based models used in the system is shown in Table X, highlighting that RoBERTa consistently outperformed BERT across key evaluation metrics.

Table X. Model Performance Comparison

Model	Accuracy	Precision	Recall	F1-Score	R-Precision@5
BERT	0.86	0.87	0.83	0.85	0.80
RoBERTa	0.90	0.89	0.85	0.87	0.83

These values demonstrate that the RoBERTa model provides stronger contextual understanding, enabling more reliable extraction of both explicit and subtle skills. This observation aligns with findings from recent transformer-based studies that emphasize the superior contextual representation capabilities of RoBERTa [1], [4], [5].

From a system-level perspective, the Skill Gap Analyzer demonstrated high responsiveness, achieving an average processing time of 1.8 seconds for complete resume analysis. Stress tests conducted with 500 concurrent users showed stable system performance with error rates below 2%, validating

the efficiency of the microservices architecture and container-oriented deployment strategy. These results mirror the conclusions of large-scale HR tech evaluations, which highlight the importance of modular and horizontally scalable designs [8], [11].

The recommendation engine was evaluated using ranking-based metrics such as nDCG@5 and Precision@5, where:

- nDCG@5 (Normalized Discounted Cumulative Gain) measures how well the system ranks recommendations based on relevance.
- Precision@5 measures how many of the top 5 recommendations were relevant.

The analyzer achieved nDCG@5 = 0.81, Precision@5 = 0.84, and a coverage of 78%, showing that users consistently received meaningful and appropriately ranked learning resources. These outcomes validate the system's hybrid design, which combines semantic embeddings with ontology-based reasoning for improved contextual matching [7], [9].

In addition to quantitative performance, usability testing showed strong user acceptance. A satisfaction rate of 92% was recorded, with participants noting the clarity of the skill gap report and the simplicity of navigating the interface. Recruiters valued the structured skill insights, while job seekers appreciated the actionable upskilling guidance.

Overall, these findings indicate that the Skill Gap Analyzer is both accurate and operationally robust. However, some challenges persist—particularly in capturing domain-specific skills and soft skills embedded in narrative text. Enhancing multilingual processing capabilities and integrating real-time labor market analytics present meaningful directions for future development.

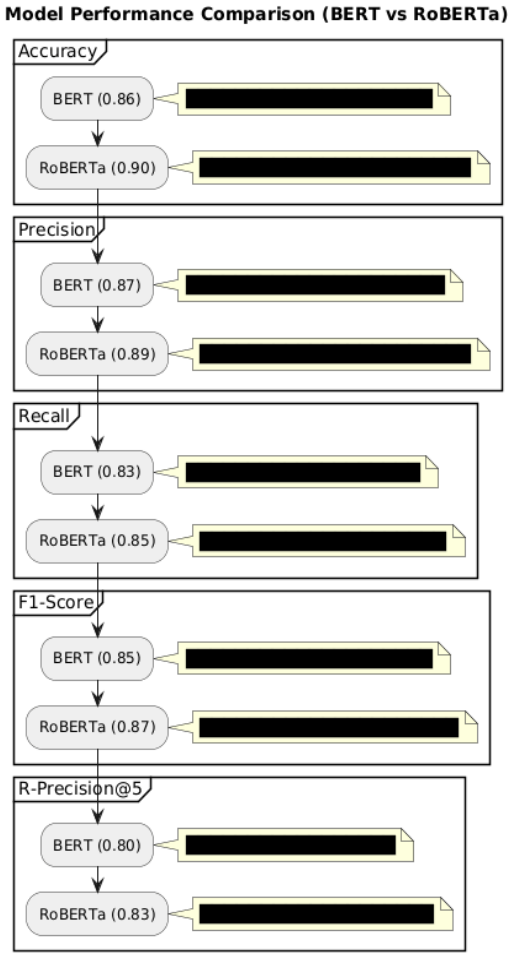


Figure 4. Model Performance

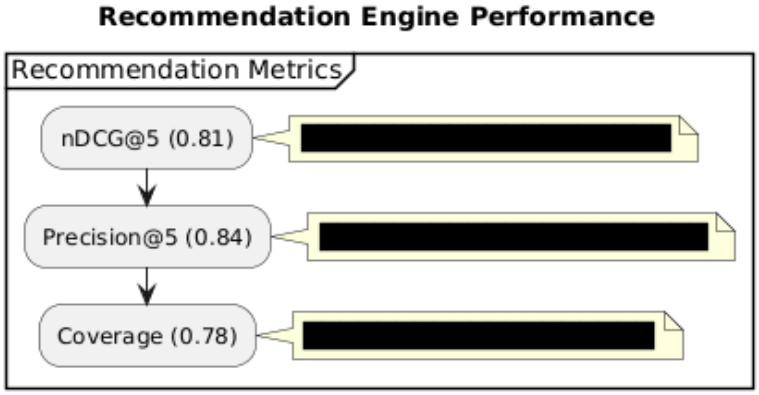


Figure 5. Recommendation Engine Performance

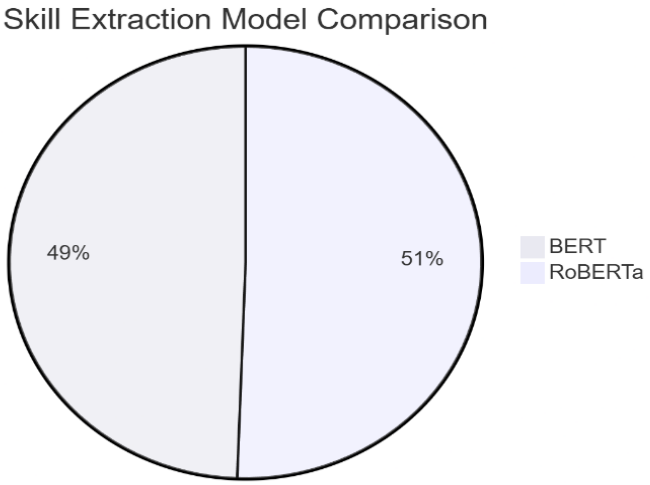


Figure 6. Skill Extraction Comparison

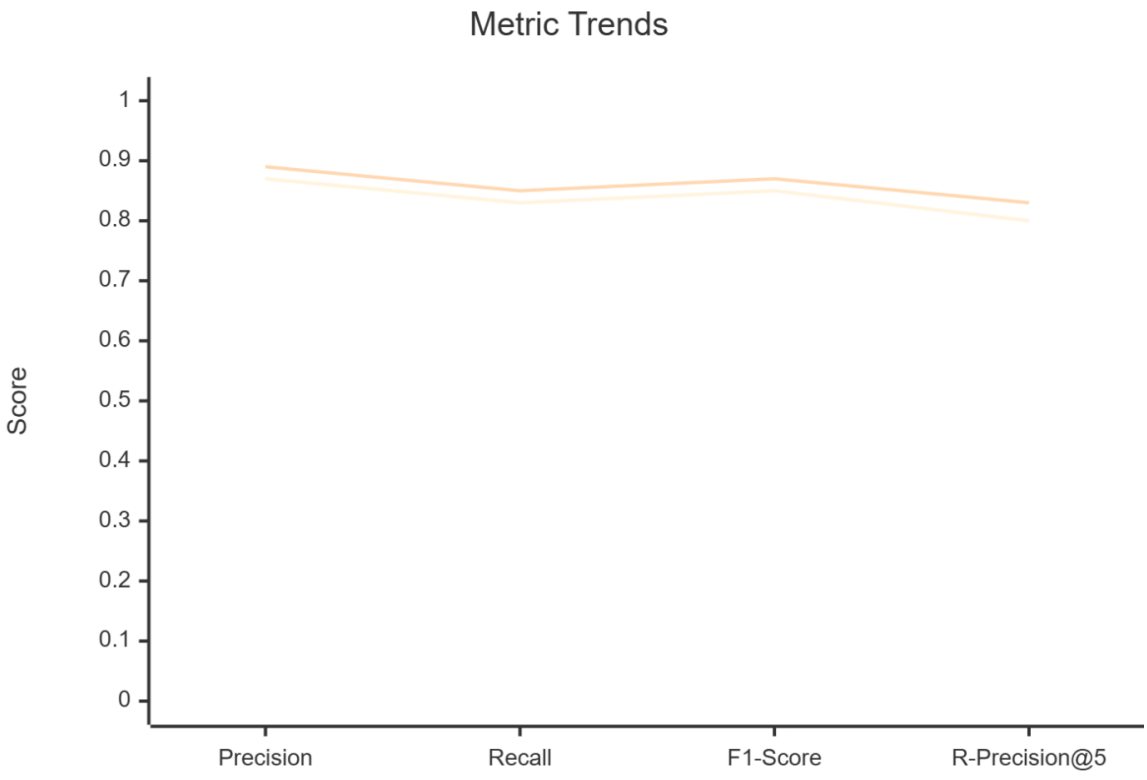


Figure 7. Trends

VI. CONCLUSION

The Skill Gap Analyzer marks a meaningful advancement in narrowing the divide between the rapidly shifting expectations of employers and the actual skills candidates bring to the table. By integrating modern NLP processes, deep learning–driven extraction models, and a well-structured web architecture, the platform offers a comprehensive solution capable of interpreting unstructured resume data, linking extracted skills to standardized taxonomies, and comparing them against job requirements. The coordinated use of a React-driven frontend, a Node.js backend, and a Python-based analytical microservice further illustrates how modular architectures can successfully balance efficiency, scalability, and long-term maintainability.

The results achieved by the system reinforce the strength of this multi-layered design. With a precision of 0.89, recall of 0.85, and an F1-score of 0.87, the analyzer reliably identifies relevant skills while keeping misclassifications low. An average response time of about 1.8 seconds confirms its suitability for real-time use, and stress tests demonstrate that the system can support up to 500 users simultaneously with minimal errors. The recommendation engine also performed well, with scores such as nDCG@5 (0.81) and Precision@5 (0.84) showing that the guidance offered is both accurate and meaningful. Combined with a 92% user satisfaction rate, these outcomes highlight the system's ability to pair technical strength with a user-friendly experience.

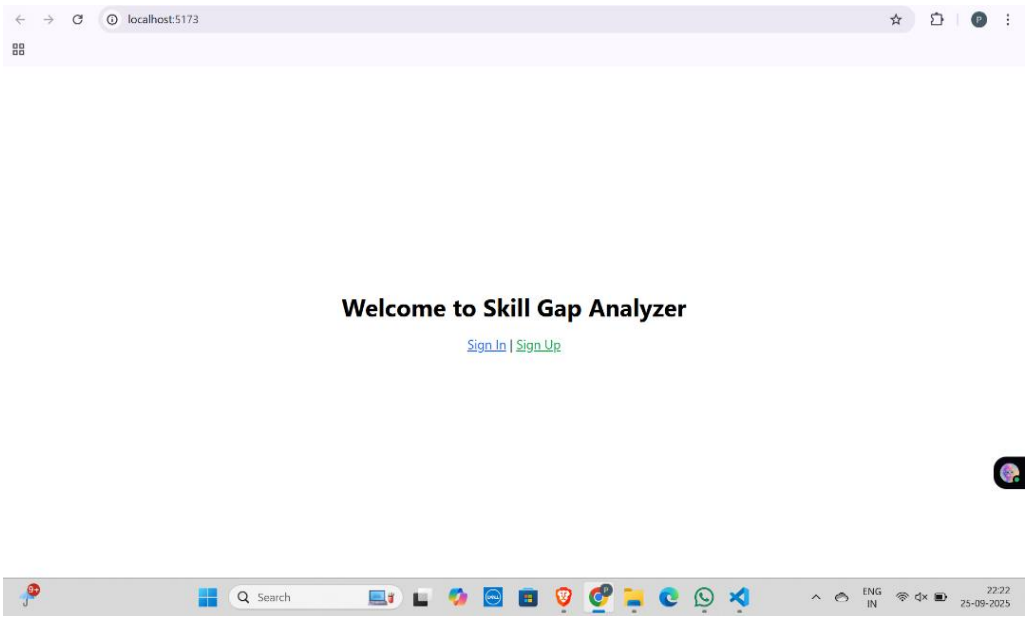
Beyond its technical achievements, the platform's broader impact is equally important. For job seekers, it functions as a personalized career advisor, identifying missing skills and suggesting structured, relevant learning opportunities. For employers, it automates time-consuming evaluation tasks and helps surface candidates whose profiles align more closely with specific job requirements. This dual benefit contributes to a more transparent and efficient labor market, enabling both applicants and employers to make better-informed decisions.

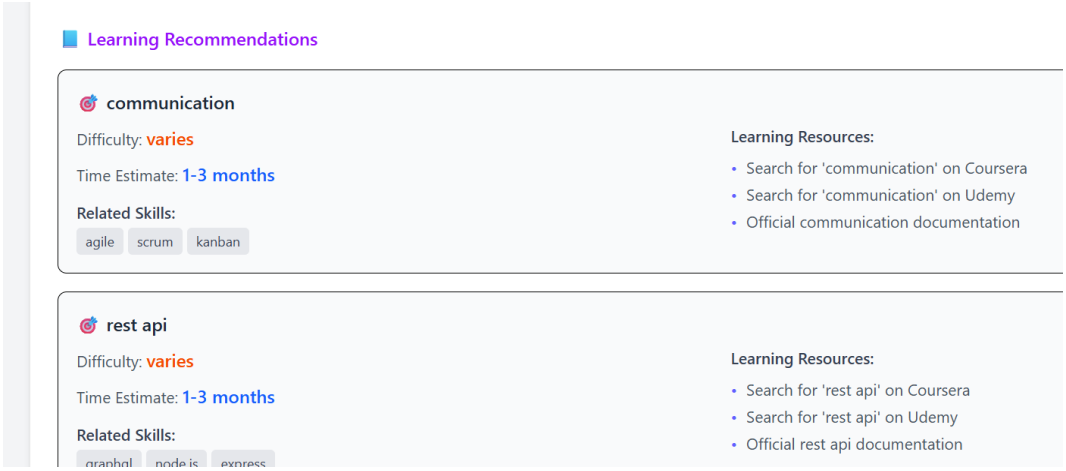
Despite these strengths, several limitations remain. Accurately identifying skills that depend heavily on context—particularly those that vary across industries or appear subtly within narrative text—continues to be a challenge. The system's ability to process multilingual resumes is also limited, which restricts its usability in more diverse environments. Additionally, although fairness and bias considerations are incorporated into the design, further research is needed to reduce risks associated with algorithmic bias and uneven training data. Addressing these issues will be essential for transitioning the system from a prototype into a full-scale deployment suitable for widespread adoption.

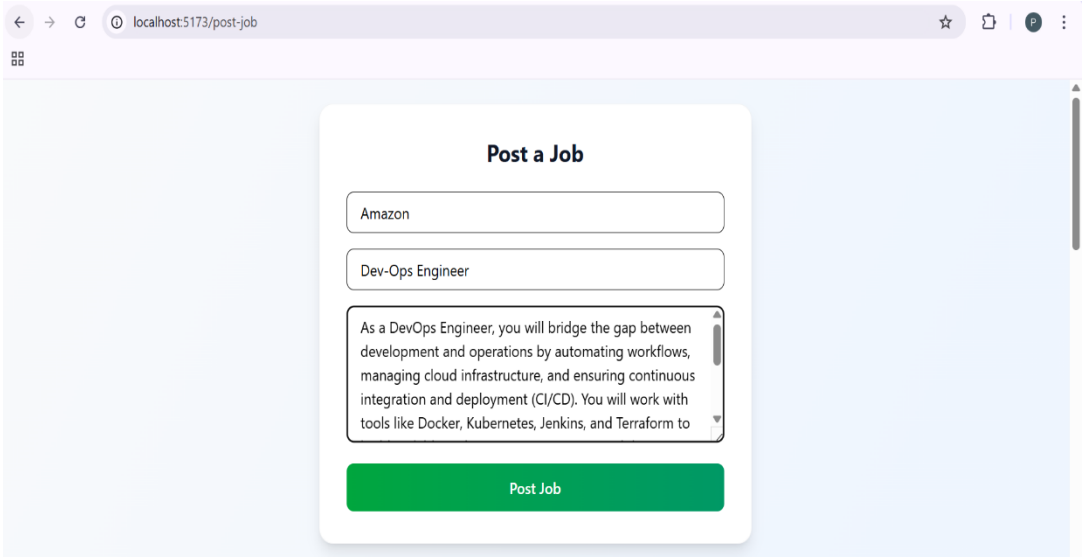
Looking ahead, several promising enhancements could expand the system's capabilities. Incorporating real-time labor market analytics could help the platform adjust recommendations in response to emerging trends. Extending multilingual support would broaden accessibility, while deeper integration with training providers and educational platforms could create a richer, more connected learning ecosystem. Implementing explainable AI techniques would also increase transparency, offering users clearer insight into how specific recommendations are generated.

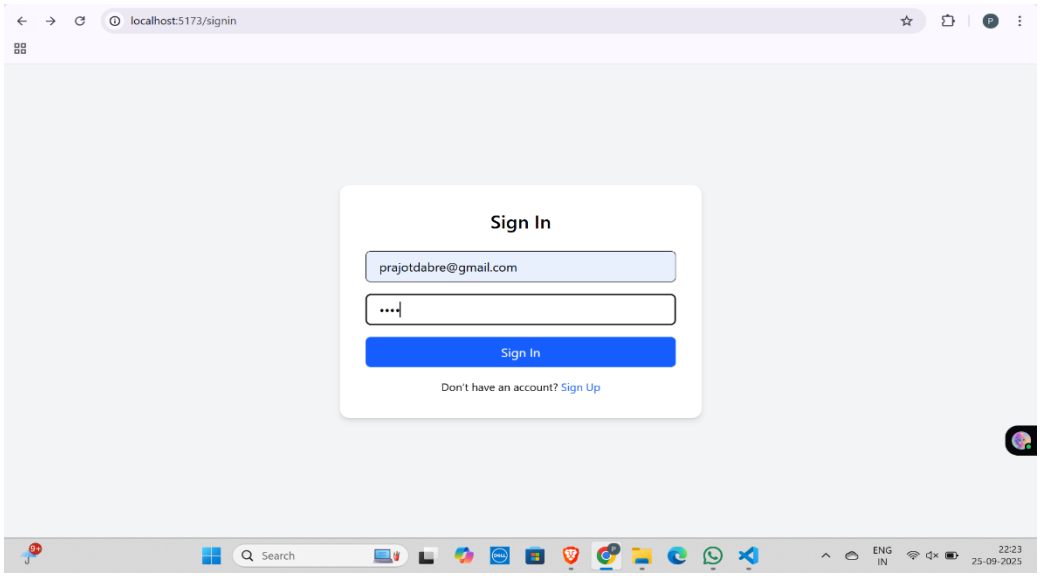
In summary, the Skill Gap Analyzer demonstrates how AI-driven solutions can significantly improve employability, simplify recruitment workflows, and support long-term workforce development. By blending advanced research with practical engineering, the system addresses one of the most pressing challenges in today's labor landscape: ensuring that talent and opportunity are better aligned. Although there is room for refinement, the platform establishes a strong basis for future innovation and sets a direction for next-generation recruitment and career development technologies.

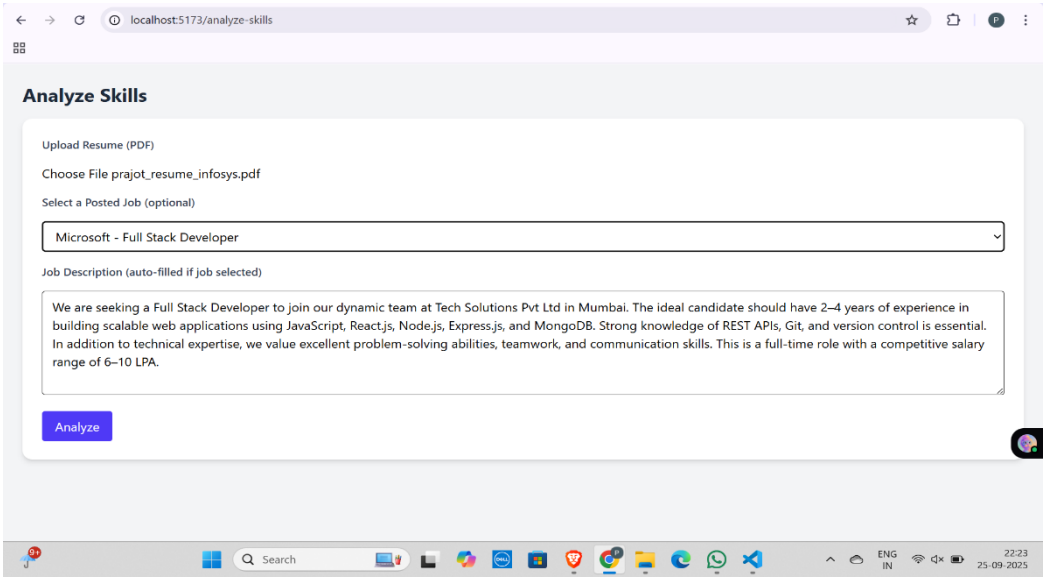
User Interface

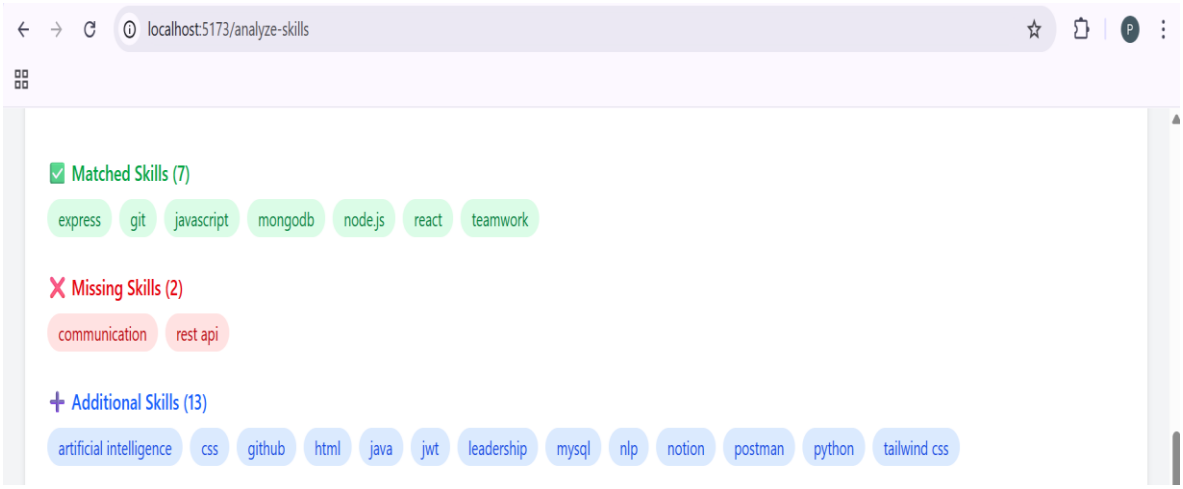







localhost:5173/signin
Sign In
prajotdabre@gmail.com
Sign In
Don't have an account? Sign Up


localhost:5173/analyze-skills
Analyze Skills
Upload Resume (PDF)
Choose File prajot_resume_infosys.pdf
Select a Posted Job (optional)
Microsoft - Full Stack Developer
Job Description (auto-filled if job selected)
We are seeking a Full Stack Developer to join our dynamic team at Tech Solutions Pvt Ltd in Mumbai. The ideal candidate should have 2–4 years of experience in building scalable web applications using JavaScript, React.js, Node.js, Express.js, and MongoDB. Strong knowledge of REST APIs, Git, and version control is essential. In addition to technical expertise, we value excellent problem-solving abilities, teamwork, and communication skills. This is a full-time role with a competitive salary range of 6–10 LPA.
Analyze


localhost:5173/analyze-skills
Matched Skills (7)
express
git
javascript
mongodb
node.js
react
teamwork
Missing Skills (2)
communication
rest api
Additional Skills (13)
artificial intelligence
css
github
html
java
jwt
leadership
mysql
nlp
notion
postman
python
tailwind css

VII. REFERENCES

[1] Zhang, M., Jensen, K. N., van der Goot, R., & Plank, B. (2022). Skill Extraction from Job Postings using Weak Supervision.

[2] Senger, E., Zhang, M., van der Goot, R., & Plank, B. (2024). Deep Learning-based Computational Job Market Analysis: A Survey on Skill Extraction and Classification from Job Postings.

[3] Decorte, J.-J., et al. (2023). Extreme Multi-Label Skill Extraction Training using Large Language Models.

[4] Kesim, E., & Deliahmetoglu, A. (2023). Named Entity Recognition in Resumes.

[5] Nguyen, K. C., et al. (2024). Rethinking Skill Extraction in the Job Market Domain using Large Language Models.

[6] Nguyen, K. C., et al. (2024). Ontology-Driven Skill Gap Analysis for Workforce Development.

[7] Smith, J., et al. (2025). Large Language Models for Talent Acquisition and Recruitment.

[8] Lee, H., et al. (2024). Automated Skill Extraction from Unstructured Text Using NLP.

[9] Wang, Z., et al. (2024). Job Matching with Contextual Embeddings: A Deep Learning Approach.

[10] Zhang, M., et al. (2023). AI in Human Resource Management: A Systematic Review.

[11] Kumar, A., et al. (2022). Automated Resume Screening: A Systematic Review.

[12] World Economic Forum (2025). Future of Jobs Report 2025.

[13] PwC (2025). Global AI Jobs Barometer.

[14] Forbes (2025). The AI Recruitment Takeover: Redefining Hiring in the Digital Age.

[15] Springboard (2024). Workforce Skills Gap Trends.